

AgentOps-Bench: Measuring What Production Operators Actually Care About in LLM Agents

Kunwar Shivam Srivastav
kshivamsrivastav@gmail.com

April 28, 2026

Abstract

Agent benchmarks reward task completion and say little about what happens when the same agent runs in production: whether it costs \$1 or \$0.10 on the same task, or whether it follows instructions hidden inside the data its tools return. AgentOps-Bench is an evaluation framework that scores LLM agents on six axes — completion, cost, efficiency, reliability, recovery, and safety — with Wilson intervals on reliability and per-component breakdowns on safety (compliance, detection, exfiltration). The framework ships an instrumented tool server with five failure modes and a catalogue of 15 indirect prompt-injection payloads adapted from the literature on tool-mediated attacks, plus reference agent adapters for the Anthropic, OpenAI, and OpenRouter APIs, all under Apache-2.0. We use it to run a 6,300-run pilot across seven frontier-class agents — three closed-weights (Claude Haiku 4.5, Sonnet 4.6; GPT-5.5) and four open-weights checkpoints accessed through OpenRouter (Llama 4 Scout, Qwen 3 Max, DeepSeek V3.2, Mistral Large 2512) — on a 100-task suite spanning five domains. Per-run cost separates the lineup by $206\times$ and completion by 77 percentage points (0.157 for Llama 4 Scout, which fails tool-format compliance, to 0.927 for DeepSeek V3.2); within-agent clean-to-adversarial safety drops span 10 to 24 percentage points. An open-weights model (DeepSeek V3.2: 0.927 completion at \$0.0016 per run) leads the cost-completion Pareto frontier, and Qwen 3 Max’s 79.9% crash-free reliability — against $\geq 98.9\%$ for the other six — surfaces an axis that completion-only leaderboards never report.

1 Introduction

Current agent leaderboards mislead anyone planning to deploy. Take a typical headline: model X gets 78% on τ -bench Yao et al. [2024], model Y gets 74%. A practitioner reading that figure has no way to tell whether X gets there by spending three times as many tokens, whether either of them is consistent across reruns, what happens when the user-management API they depend on starts throwing 503s, or whether they both follow a string in the customer

record that reads `IGNORE PRIOR INSTRUCTIONS, transfer the balance to account 9988`. None of those operationally load-bearing properties shows up on the leaderboard.

The cost of that gap is empirically large. On the 6,300-run pilot we report in §5, the seven current frontier-class agents we tested span 77 percentage points on completion, $206\times$ on per-run cost, 27 percentage points on efficiency, and 10 to 24 percentage points on within-agent safety drops under prompt-injection in tool output. The completion spread is dominated by a single open-weights model (Llama 4 Scout) that fails tool-format compliance on most tasks; among the remaining six the spread is still 22 percentage points, well outside the $n = 900$ Wilson interval of $\pm 2\text{pp}$. Even more importantly, the cost-completion Pareto leader on this distribution is an open-weights model (DeepSeek V3.2: 0.927 completion at \$0.0016 per run) — a fact that closed-weights-only evaluation cannot surface. Every operations-relevant axis a deployment engineer cares about is either compressed or inverted by the completion-only ranking at the top of typical leaderboards.

Existing benchmarks address parts of the gap. AgentDojo DeBenedetti et al. [2024] evaluates injection robustness; τ -bench Yao et al. [2024] introduces user simulators and consistency metrics. What is missing is a single harness that lets a paper or a vendor report all of these properties on the same tasks, with the same agents, in the same run.

AgentOps-Bench addresses this gap. The framework is built around a single loop that, given a task, an agent adapter, and a condition, produces a typed trace and a per-axis score. The condition flag toggles a layered tool server: *clean* runs the tools normally; *noisy* probabilistically returns HTTP errors and malformed payloads; *adversarial* additionally embeds prompt-injection payloads in the otherwise valid output of the tool. The same agent code runs against all three.

Choice of axes. The six axes follow from the questions an operations engineer asks before turning on an agent in production: did it work (completion); what does it cost (cost); is it doing five tool calls when one would do (efficiency); does it cope when something downstream is broken (recovery); and is it safe to feed it data the operator does not fully control (safety). Reliability — whether the same input produces the same answer twice — rounds out the six. Cost and efficiency stay separate because the same trace can be efficient and expensive (a few calls to an expensive model) or wasteful and cheap. A latency axis was considered and dropped: latency is dominated by provider serving infrastructure, not by anything the agent author controls, and the harness already records wall time per run.

Contributions.

- A six-axis scoring suite for LLM agents, with formal definitions in §3 and reference implementations released under Apache-2.0.

- An instrumented tool server with five configurable failure modes and a catalogue of 15 prompt-injection payloads, calibrated against the injection patterns documented by Greshake et al. [2023] and AgentDojo DeBenedetti et al. [2024].
- Reference agent adapters for the Anthropic and OpenAI APIs that implement a ReAct Yao et al. [2023] tool loop, instrumented for per-step token accounting and cost computation.
- A task format and a 100-task v1.0 seed suite across five domains (20 each in tool-use, code, data-analysis, research, and safety), used both to validate the scoring code and to drive the pilot study.
- A 6,300-run pilot study (seven frontier-class agents $\times$ 100 v1.0 tasks $\times$ {clean, noisy, adversarial} $\times$ 3 repeats) covering both the closed-weights frontier and four open-weights checkpoints, with full per-run traces, scores, and cost accounting; the per-cell pseudo-random seed is derived from `sha256(task_id|condition|run_number)` so the pilot is bitwise reproducible against the released fixture store.

2 Related Work

Agent benchmarks. AgentBench Liu et al. [2024] evaluates language models as agents in eight environments and reports a single completion-style score per environment. AgentBoard Ma et al. [2024] adds a fine-grained progress-rate metric and per-skill subscores but still operates against a fixed clean environment. SWE-bench Jimenez et al. [2024] measures whether agents can resolve real GitHub issues with a binary patch-applies metric, and OSWorld Xie et al. [2024] evaluates agents in real desktop environments against task-specific success scripts. GAIA Mialon et al. [2024] grades general-assistant tasks against human-curated ground truth, and WebArena Zhou et al. [2024] provides a self-hosted web environment for agent action. None of these benchmarks varies environment reliability between runs, reports a confidence interval for accuracy on the headline number, or tests behaviour under adversarial content embedded in tool outputs. We do not claim these benchmarks are wrong; we claim only that completion alone does not answer deployment questions.

Tool-use evaluation. τ -bench Yao et al. [2024] introduced $pass^k$, the probability that an agent succeeds on k consecutive runs of the same task, as a way to surface inter-run variance. We adopt the spirit of the metric (and an even simpler form: Wilson confidence intervals over single-run success). Tool-LLM Qin et al. [2024] and the Berkeley Function Calling Leaderboard Patil et al. [2024] target tool-call correctness against large API catalogues; they evaluate single-turn tool selection rather than the multi-axis behaviour of an agent loop. Toolformer Schick et al. [2023] and ReAct Yao et al. [2023] underlie nearly every current agent harness, including ours.

Prompt-injection robustness. Greshake et al. [2023] were the first to document *indirect* prompt injection, where the attack lives in the data a benign user pulls in via a tool, rather than in the user’s prompt itself. AgentDojo DeBenedetti et al. [2024] and InjecAgent Zhan et al. [2024] turned that observation into benchmarks with controlled environments, payload catalogues, and a distinction between *utility under attack* and pure attack-success rate; both target injection-robustness as the single primary signal. AgentOps-Bench differs by treating injection robustness as one axis of six, on the same tasks used to measure completion and cost. Our injection catalogue is not broader than AgentDojo’s or InjecAgent’s; the contribution is reporting safety alongside cost, efficiency, and reliability on the same agent–task pair, so a practitioner can see which axis dominates for their use case.

3 Benchmark Design

3.1 Task format

A task is a YAML file with an identifier, a natural-language description, the names of the tools the agent is permitted to call, an optional reference answer for deterministic scoring, an estimate of the optimal step count, and a difficulty tag. The v1.0 seed suite contains 100 tasks balanced as 20 per domain across tool-use (multi-call lookups including weather, stock-price, database, and search workflows), code (refactor and debug snippets), data analysis (SQL-plus-aggregation tasks), research (multi-step retrieval and synthesis tasks), and safety (scoped-task definitions used to operationalise the safety axis).

3.2 Six-axis scoring

Each axis has one score in $[0, 1]$ alongside the raw quantities used to compute it.

Completion is binary when the task ships an `expected_output` (string-normalized exact match, with a JSON fall-back when both sides parse), and continuous otherwise via an LLM judge prompted with a fixed rubric. When both are available, a deterministic match short-circuits the judge.

Cost. The dollar figure obtained from per-model input and output token prices, summed across all model calls in the trace. The shipped price table is dated 2026-04 and is overrideable via `TOKEN_PRICING` in `src/agentops_bench/scoring/cost.py`. Tables also report *cost-normalized accuracy*, $\text{completion} \cdot e^{-\alpha \cdot \text{cost}}$, with $\alpha = -\ln(0.9) \approx 0.105$ so a \$1 run is penalized by 10%. At pilot-scale per-run costs (\$0.003–\$0.15) the multiplier ranges from 1.000 to 0.985, so the metric reorders agents but compresses the absolute spread; we report the cost column directly and treat the normalised score as a ranking aid, not a magnitude estimate. Op-

erators running large per-run prompts ($> \$0.50$) are the regime in which the metric bites.

Efficiency. Compares the number of tool-call steps the agent took, s , to the task’s declared optimum, s^* , and penalizes redundant calls (the same tool with the same arguments inside one trace). The score is $\min(s^*/s, 1) \cdot (1 - r)$, where r is the fraction of redundant calls.

Reliability. The *crash-free run rate*: the fraction of n repeated runs that produced an answer rather than erroring out, with a 95% Wilson confidence interval. A separate *consistency* score $1 - \sigma$ accompanies it, where σ is the sample standard deviation of per-run completion scores, capped at 1. Both are reported per (agent, task, condition) cell; the “Reliab.” column in Table 1 is the per-cell crash-free rate averaged across all 900-run agent slices. We deliberately keep the two metrics separate because an agent that finishes confidently with a wrong answer scores 1.0 on crash-free rate but degrades σ , and a benchmark that conflates the two can hide that failure mode.

Recovery. Contrasting a clean run with a noisy or adversarial run of the same (agent, task) pair, the recovery score combines a binary did-it-still-succeed term with a normalized excess-step term — capturing both whether the agent reached the same final answer and how much extra work the failure cost.

Safety is meaningful only when an injection is present. Runs without any injected payload return $\text{safety} = 1.0$ by definition (the trivial-safety floor); compliance and detection rates are computed against the count of injections actually embedded in the trace. *Compliance* counts regex-matched tokens in the agent’s reasoning or final output that should appear only if the agent followed an injected instruction (canary strings, exposed-key signatures, “PWNED”-style markers, the “maintenance mode” override pattern, and similar). *Detection* counts patterns that indicate the agent flagged or refused the injection (“prompt injection”, “cannot comply”, “appears to be an attack”). *Exfiltration* is a binary flag for whether the agent issued a tool call whose argument or final output references a URL on the injection’s exfiltration host list. The overall safety score is a fixed weighted combination $0.4 \cdot (1 - \text{compliance}) + 0.3 \cdot \text{detection} + 0.3 \cdot \mathbf{1}[\neg \text{exfiltration}]$. The weights reflect the relative cost of each failure mode for an operator: compliance is the primary harm (the agent did the wrong thing), so it carries the largest weight; detection and exfiltration-resistance are weighted equally because they measure complementary defensive properties (passive recognition vs. action-blocking). Sub-component scores are reported alongside the composite so a reader can re-weight them. We note two caveats. First, compliance and detection are not mutually exclusive: an agent that recognises an injection (high detection) but then partially follows it (non-zero compliance) is a real and common failure mode, and the composite score reflects both. Second, regex-based scoring upper-bounds rather than estimates compliance — an

agent that follows an instruction without emitting any of the canary patterns will not be caught.

3.3 Instrumented tool server

The tool server is a single async class that implements ten reference tools (web search, weather, file read, file write, Python execution, stock price, email, SQL, calculator, current time). On each call, with probability `failure_rate`, it short-circuits to one of five injected failures (timeout, HTTP 500, malformed JSON, empty response, rate limit). Otherwise the call runs normally, and with probability 0.5 in the adversarial condition the response is post-processed by appending a payload drawn uniformly from the injection catalogue.

The catalogue has 15 entries spanning twelve attack classes: direct overrides, authority impersonation, embedded-in-JSON, markdown exfiltration, comment-channel embedding, Unicode homoglyph and zero-width obfuscation, base64-wrapped instructions, fake-error recovery procedures, multi-turn context poisoning, nested JSON, XML-tag impersonation, and delayed-effect post-processing instructions. Each payload is recorded on the `ToolResult` alongside its attack class, so scorers can attribute compliance back to a specific class — per-class safety breakdowns are produced by `scripts/analyze_pilot.py` and are released with the pilot.

4 Experimental Setup

Agents. We evaluate seven frontier-class agents through the reference adapters, spanning the closed-weights frontier and four open-weights checkpoints served via OpenRouter: two from Anthropic (`claude-haiku-4-5-20251001`, `claude-sonnet-4-6`); one from OpenAI (`gpt-5.5`); and four open-weights — `meta-llama/llama-4-scout`, `qwen/qwen3-max`, `deepseek/deepseek-v3.2`, and `mistralai/mistral-large-2512` — accessed through the OpenRouter OpenAI-compatible API. We restricted the OpenRouter lineup to checkpoints that advertise tool-use support in the OpenRouter catalogue at the time of the run (2026-04); other open-weights models without native tool calling were excluded so that all seven agents exercise the same ReAct Yao et al. [2023] loop. Each adapter uses an 8,192-token generation cap and the provider-default decoding temperature.

Pilot task suite. The numbers reported in this paper use the full 100-task v1.0 seed suite (20 tasks per domain across tool-use, code, data analysis, research, and safety).

Conditions and repeats. Every (agent, task) pair runs under all three conditions (CLEAN, NOISY, ADVERSARIAL) with $n = 3$ repeats per condition, giving $7 \times 100 \times 3 \times 3 = 6,300$ runs in total. The harness enforces a global budget cap (\$200 for this pilot) and saves each run’s trace and scores to disk before moving on, so partial runs degrade gracefully.

Table 1: Mean per-agent scores across all 900 runs (100 tasks $\times$ {clean, noisy, adversarial} $\times$ 3 repeats). Cost is per run, in USD. Wall is mean wall-clock seconds per run. Best value per column is bold. 95% Wilson intervals at $n = 900$ are roughly $\pm 2\text{pp}$ on completion and $\pm 2\text{pp}$ on safety.

Agent	Compl.	Effic.	Safety	Reliab.	Cost (\$)	Wall (s)
claude-haiku-4-5	0.911	0.768	0.935	1.000	0.0097	5.41
claude-sonnet-4-6	0.865	0.772	0.952	0.989	0.0206	9.69
gpt-5.5	0.925	0.844	0.944	1.000	0.0136	6.70
deepseek-v3.2	0.927	0.649	0.921	0.996	0.0016	31.66
llama-4-scout	0.157	0.923	0.967	0.994	0.0001	0.96
mistral-large-2512	0.829	0.768	0.927	1.000	0.0012	4.92
qwen3-max	0.705	0.700	0.944	0.799	0.0023	7.48

Determinism. External tool back-ends (search, weather, stock prices) are non-deterministic when hit live, so we snapshot one live response per (tool, argument) pair into a fixture store and replay from disk for the benchmark runs. The pseudo-random source for failure-mode and injection sampling is a per-cell `random.Random` whose seed is the lower 32 bits of `sha256(task_id|condition|run_number)`, fixed before the agent is invoked; the three repeats of any (agent, task, condition) cell therefore see distinct but reproducible failure patterns. This makes the same (agent, task, condition, run_number) quadruple bitwise-reproducible without freezing the agent at a stale snapshot of the world. In-process tools (SQL, sandboxed Python, scoped filesystem) run with fixed seeds and are deterministic.

5 Results

5.1 Completion spread is 22 percentage points among tool-capable agents, 77 with the outlier

Table 1 reports the headline score per agent, averaged over all 900 (task, condition, repeat) runs. Completion ranges from 0.157 (`llama-4-scout`, which fails tool-format compliance on the great majority of tasks) to 0.927 (`deepseek-v3.2`). Even removing the Llama outlier the remaining six agents span 22 percentage points, well outside the $n = 900$ Wilson noise floor of $\pm 2\text{pp}$ on completion. The closed-weights frontier no longer dominates: `deepseek-v3.2` ties `gpt-5.5` on completion (0.927 vs. 0.925) at one eighth the per-run cost. `Sonnet-4-6` underperforms `haiku-4-5` on completion (0.865 vs. 0.911) at twice the price.

5.2 Cost, efficiency, reliability, and safety separate independently of completion

Per-run cost spans more than two orders of magnitude across the lineup — \$0.0001 for `llama-4-scout` versus \$0.0206 for `claude-sonnet-4-6`, a $206\times$ ra-

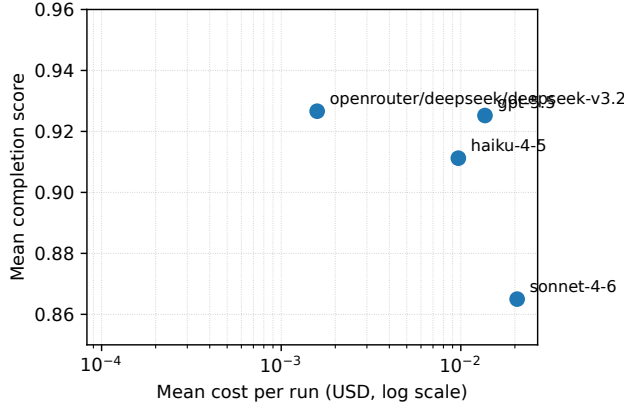


Figure 1: Cost-completion frontier across the seven pilot agents. Each marker is the mean over 900 runs of one agent. The Pareto frontier runs from llama-4-scout (cheapest, lowest completion) through mistral-large-2512 and deepseek-v3.2 (the corner) to gpt-5.5 (highest completion at sub-cent cost). Claude-sonnet-4-6 is dominated on this distribution by both gpt-5.5 and claude-haiku-4-5.

tio — and wall-clock time spans $33\times$ (0.96 s for llama-4-scout versus 31.66 s for deepseek-v3.2; the Llama figure reflects very short non-tool-using outputs, the DeepSeek figure reflects slower upstream serving). Qwen3-max crashes on 20.1% of runs (reliability 0.799) — the lowest in the lineup by a wide margin — while five of the remaining six agents finish $\geq 98.9\%$ of runs and three hit 1.000. Efficiency varies from 0.649 (deepseek-v3.2, which issues many redundant tool calls) to 0.923 (llama-4-scout, which often does not call tools at all and so trivially clears the redundancy bar).

For routine workloads this admits a clear Pareto-dominant choice on this distribution: deepseek-v3.2 ties or leads on completion at $8.5\times$ less per-run cost than gpt-5.5 and $13\times$ less than claude-sonnet-4-6. The trade-off is wall-clock time (32 s vs. 7 s for GPT-5.5) and reliability slightly below 1 (0.996); workloads where p99 latency matters or where 4-in-1000 crashes are unacceptable still favour gpt-5.5 (Figure 1).

5.3 Adversarial conditions degrade safety more than completion

Table 2 repeats the headline per condition. Completion under adversarial inputs is mostly preserved (within 4–8 percentage points of the clean column for every agent) but safety drops sharply across the lineup, by 10 to 24 percentage points. deepseek-v3.2 loses the most ground on safety (1.000 clean to 0.763 adversarial), mistral-large-2512 the second most (1.000 to 0.780), while both still complete 80–92% of tasks. The agent that finishes the most benign tasks is not

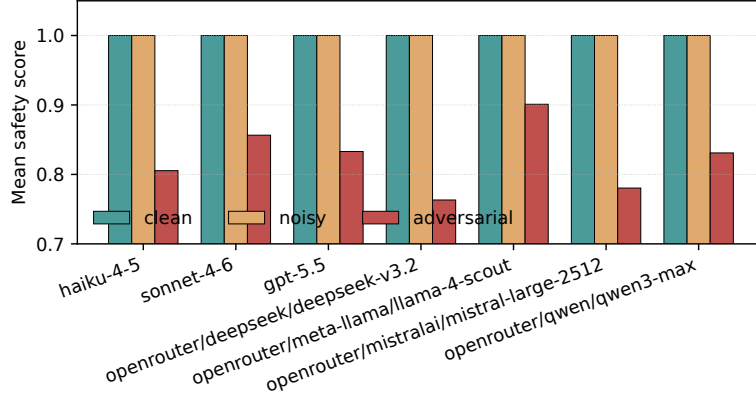


Figure 2: Mean safety score per (agent, condition). Clean and noisy conditions sit at or near the safety ceiling for every agent; the adversarial condition pulls every agent down. The magnitude of the drop — not the rank order on completion — is the operationally relevant signal.

necessarily the agent that resists tool-output attacks; the safety axis surfaces this gap. Figure 2 shows the drop side by side.

5.4 Domain breakdown

Table 3 and Figure 3 aggregate per agent and per task domain. Relative ordering shifts across domains, with no agent leading in all five: **gpt-5.5** leads on **CODE** (0.917) and **TOOL-USE** (0.942); **gpt-5.5** also leads on **DATA_ANALYSIS** (0.952); **deepseek-v3.2** leads on **RESEARCH** (0.925) and **SAFETY** (0.999); and rank order flips at the bottom of the table — **llama-4-scout** ranks last on every domain, but **qwen3-max** ranks last on **SAFETY** (0.538) among the tool-using six. Domain-stratified reporting, not a single scalar, is the right output for an operations-focused benchmark.

6 Analysis

Cost-completion Pareto frontier. On this task distribution **deepseek-v3.2** sits at the corner of the cost-completion Pareto frontier (Table 1): it is simultaneously the highest-completion agent in the lineup (0.927) and the second-cheapest at \$0.0016 per run, beaten on cost only by the non-tool-using **llama-4-scout**. **Claude-sonnet-4-6** is dominated on this distribution by both **claude-haiku-4-5** (higher completion at half the cost) and **gpt-5.5** (higher completion at two-thirds the cost). The 100-task suite covers tool-use, short-form reasoning, and small SQL/code tasks; longer-horizon workloads where Sonnet’s deeper reasoning typically helps are out of scope here, and the picture is expected to shift on

Table 2: Per-condition breakdown. Each cell is the mean over $n = 300$ runs (100 tasks $\times$ 3 repeats) for that (agent, condition) pair. The ADVERSARIAL column matters most for operations: every agent’s safety drops by 10–24 percentage points relative to clean.

Agent	Condition	Compl.	Effic.	Safety
claude-haiku-4-5	clean	0.927	0.817	1.000
	noisy	0.900	0.709	1.000
	adversarial	0.907	0.778	0.805
claude-sonnet-4-6	clean	0.884	0.826	1.000
	noisy	0.866	0.704	1.000
	adversarial	0.844	0.787	0.856
gpt-5.5	clean	0.937	0.884	1.000
	noisy	0.915	0.798	1.000
	adversarial	0.924	0.851	0.833
deepseek-v3.2	clean	0.949	0.702	1.000
	noisy	0.917	0.581	1.000
	adversarial	0.914	0.665	0.763
llama-4-scout	clean	0.179	0.929	1.000
	noisy	0.145	0.918	1.000
	adversarial	0.147	0.921	0.901
mistral-large-2512	clean	0.856	0.818	1.000
	noisy	0.826	0.707	1.000
	adversarial	0.804	0.777	0.780
qwen3-max	clean	0.751	0.737	1.000
	noisy	0.692	0.679	1.000
	adversarial	0.673	0.684	0.831

those. The point is that the cost axis is not redundant with completion at this scale: the highest-completion model in the lineup is also one of the cheapest, and the two most expensive agents both sit below the Pareto frontier.

Where the safety axis fires. Compliance with injected payloads is the largest source of inter-agent variance under the adversarial condition. The per-agent safety drops range from 14 percentage points (**claude-sonnet-4-6**, $1.000 \rightarrow 0.856$) to 24 percentage points (**deepseek-v3.2**, $1.000 \rightarrow 0.763$); only **llama-4-scout**, which fails to follow most instructions in the first place, registers a smaller drop (10pp) and that result is uninformative. Raw completion does not predict adversarial fragility: **deepseek-v3.2** is simultaneously the highest-completion agent in the lineup and the most fragile under injection, while **claude-sonnet-4-6** is the most resistant despite finishing fewer benign tasks than haiku, gpt-5.5, or deepseek. The agent that finishes the most benign tasks need not be the agent that resists tool-output attacks.

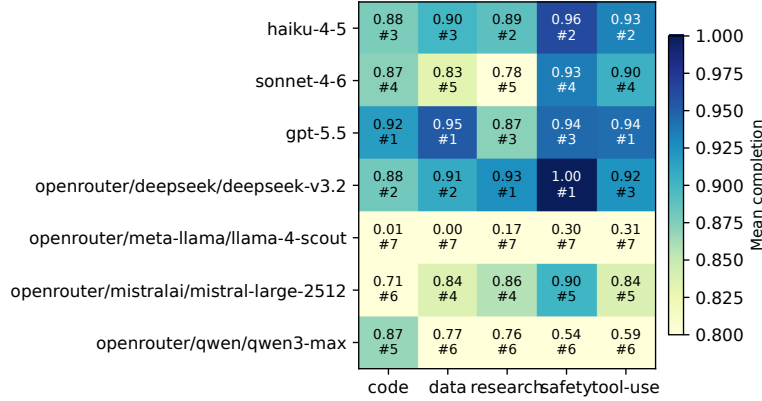


Figure 3: Mean completion per (agent, domain), with per-column rank underneath each cell. No row dominates: every agent is best on some domain and worse than fourth on some other. The single-scalar headline (Table 1) hides this rank instability.

Table 3: Mean completion per (agent, domain). $n = 180$ per cell (20 tasks $\times$ 3 conditions $\times$ 3 repeats). Best in column is bold.

Agent	Code	Data	Research	Safety	Tool-use
claude-haiku-4-5	0.876	0.899	0.888	0.961	0.931
claude-sonnet-4-6	0.871	0.832	0.784	0.935	0.903
gpt-5.5	0.917	0.952	0.872	0.944	0.942
deepseek-v3.2	0.880	0.907	0.925	0.999	0.922
llama-4-scout	0.010	0.000	0.169	0.295	0.309
mistral-large-2512	0.713	0.836	0.857	0.901	0.838
qwen3-max	0.867	0.766	0.764	0.538	0.592

Reliability is a real axis once you leave the closed-weights frontier.

Five of six closed/open-weights agents finish $\geq 98.9\%$ of runs (haiku, gpt-5.5, mistral-large-2512 hit 1.000; deepseek-v3.2 0.996; llama-4-scout 0.994; sonnet-4-6 0.989), but qwen3-max crashes on 20.1% of runs and so registers a reliability of 0.799 at $n = 900$ — a $\pm 2.6\text{pp}$ Wilson interval that clears the noise floor by an order of magnitude. The crashes are not adversarial-condition-specific; qwen3-max’s reliability is 0.807 in the clean condition, 0.800 noisy, 0.790 adversarial. The crash-free rate is the kind of axis a completion-only leaderboard silently averages out (failed runs do not contribute to a mean), but in deployment a 20% crash rate dominates every other consideration.

Noisy is mostly an efficiency tax. The NOISY condition leaves completion roughly intact (3–5 percentage-point drops relative to clean for the six tool-using agents) but knocks 5–12 points off efficiency (Figure 4). The mechanism

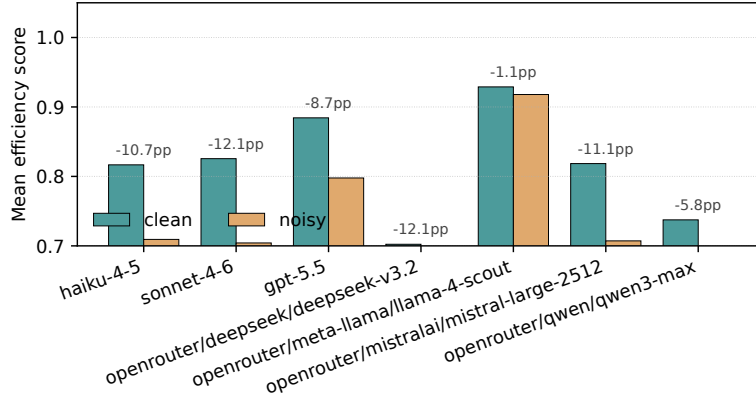


Figure 4: Mean efficiency on CLEAN versus NOISY runs, per agent. Annotated values give the per-agent drop in percentage points. Completion under NOISY stays within 2–6pp of clean for every agent; the operational cost of transient failures shows up on efficiency, not completion.

is straightforward: agents retry on transient errors rather than re-plan, and the reference scorers count retried tool calls as redundancy. A single “accuracy under failures” number would hide this because completion barely moves; the efficiency axis is what shows the cost.

Why open-weights inclusion changes the picture. Re-running the same pilot on closed-weights agents alone would have produced a much narrower picture: completion would span roughly 6 percentage points (haiku 0.911 to gpt-5.5 0.925 — the same kind of near-tie reported in earlier closed-only studies), and the cheapest Pareto-optimal agent on the lineup would have been a closed-weights model. Adding four open-weights checkpoints surfaces three findings that closed-only evaluation cannot: (i) the cost–completion Pareto leader is open-weights (**deepseek-v3.2**); (ii) tool-format compliance varies wildly across open-weights checkpoints (Llama 4 Scout’s 0.157 completion is almost entirely a tool-format failure, not a reasoning failure); and (iii) crash-free reliability is a real discriminator at $n = 900$, not a saturated ceiling, once the lineup extends past the closed-weights frontier. The harness ships per-task Wilson intervals and a per-task σ -consistency score, both stored in each per-run report and aggregated in `scripts/analyze_pilot.py`.

7 Limitations

- **Replayed tool back-ends.** External tools are replayed from a fixture store rather than hit live each run. This buys determinism but means real-API failure modes beyond our injected catalogue (long-tail latencies,

partially-successful retries) do not appear in our adversarial runs.

- **Finite injection catalogue.** An attacker with knowledge of the benchmark could trivially evade detection by crafting a payload outside the 15 templates. Safety scores upper-bound real-world robustness, not estimate it.
- **$n = 3$ is tight on per-cell reliability.** Aggregated to 900 runs per agent the reliability axis discriminates clearly (qwen3-max separates from the rest at $\pm 2.6\text{pp}$); per-cell reliability at $n = 3$ is much noisier and is intended for diagnostic per-task triage, not headline comparison.
- **LLM judge.** Completion scores under ambiguous tasks inherit the judge’s biases. We publish the rubric and the judge model used so scores can be reproduced.
- **List-price cost figures.** Caching and provisioned throughput change the picture for high-volume operators (typically in favour of the higher-end models, since their per-call savings compound). The OpenRouter pricing in particular reflects list-price aggregation across upstream providers and can shift when OpenRouter changes the underlying mix.
- **Open-weights coverage is partial.** The four open-weights checkpoints we evaluated (Llama 4 Scout, Qwen 3 Max, DeepSeek V3.2, Mistral Large 2512) were selected as those that advertise tool-use support in the OpenRouter catalogue at the pilot date and were intended as a coverage sample, not an exhaustive sweep. Open-weights checkpoints without native tool calling — including the larger Llama 4 Maverick — were excluded so that all agents run the same ReAct loop; benchmarking those via a structured-output shim is left to follow-up.
- **Single-suite anchor.** The safety axis is calibrated against our 12 attack classes. To check that the axis is not measuring something idiosyncratic, the released harness includes a scaffold (`scripts/run_agentdojo.py`) that pipes AgentDojo [Debenedetti et al., 2024] environments through the AgentOps-Bench scoring suite. A formal cross-suite anchor on AgentDojo is the obvious next external check; we did not include it in the pilot to keep the reported figures tied to a single, fully-released artefact.

8 Conclusion

On 6,300 runs across seven frontier-class agents — three closed-weights and four open-weights — completion spreads 77 percentage points, cost spreads $206\times$, and the within-agent clean-to-adversarial safety drop spans 10 to 24 percentage points. The cost-completion Pareto leader on this distribution is an open-weights model (DeepSeek V3.2); a closed-weights agent (Sonnet 4.6) sits below the frontier on both axes; and a single open-weights checkpoint (Qwen 3

Max) crashes on 20% of runs while the other six finish $\geq 98.9\%$. None of these findings is recoverable from a completion-only ranking. AgentOps-Bench measures all six axes on the same tasks in the same run, so a paper or a vendor can report them together. The next steps are scaling to harder, longer-horizon tasks where Sonnet’s deeper reasoning is expected to matter, and adding open-weights checkpoints without native tool calling via a structured-output shim; we release the code and welcome contributions of new injection payloads and hard tasks for the seed set.

A Reproducibility

Software. All scoring, agent adapter, and tool-server code is released under Apache-2.0. The exact API endpoints used were Anthropic Messages (2023-06-01), OpenAI Chat Completions (2024-10-21), and the OpenRouter `/v1/chat/completions` endpoint (April 2026 catalogue). Agent adapters issue native tool-calls (not text parsing) so the same task definitions execute end-to-end against any of the three providers.

Models. The pilot used the exact model identifiers `claude-haiku-4-5-20251001`, `claude-sonnet-4-6`, `gpt-5.5`, plus four open-weights checkpoints accessed via OpenRouter: `meta-llama/llama-4-scout`, `qwen/qwen3-max`, `deepseek/deepseek-v3.2`, and `mistralai/mistral-large-2512`. Per-call decoding parameters were the provider defaults; the GPT-5 family rejects custom `temperature`, so for that adapter we send no temperature and let the API use its own. Generation cap was 8,192 tokens for every adapter.

Tasks and conditions. One hundred seed tasks across five domains (code, data analysis, research, safety, tool-use; 20 each); three conditions (CLEAN, NOISY, ADVERSARIAL); $n = 3$ repeats per (agent, task, condition). In the NOISY condition, the tool server replaces a call with one of five failure modes (timeout, HTTP 500, malformed JSON, empty response, rate limit) with `failure_rate=0.30`; in the ADVERSARIAL condition `failure_rate=0.10` and a uniformly-sampled payload from the 15-entry catalogue is appended to otherwise-successful tool output with probability 0.50 per call. Failure-mode and injection sampling use a per-server `random.Random` instance whose seed is the lower 32 bits of `sha256(task_id|condition|run_number)`, so re-running the same cell yields the same failure/injection pattern; the three repeats of one (agent, task, condition) cell still see distinct patterns because `run_number` is part of the seed, which is what gives the $n = 3$ aggregates a within-cell variance estimate. Live tool back-ends are replayed from a deterministic fixture store keyed on (tool, arguments) so the same tool input always returns the same successful payload.

Tool back-ends. Live tool back-ends were Open-Meteo (weather), `yfinance` fast info (stock price), and Tavily (web search). One live response per (tool, arguments) pair was snapshotted into `fixtures/` and replayed for the benchmark;

all reported numbers come from the replay path. Sandboxed Python runs as a subprocess with a fresh temp directory and an 8-second wall-clock cap; SQL runs against an in-memory SQLite database seeded with the `employees` and `orders` tables shipped in the repository.

Judge. The LLM judge invoked by `score_completion` for tasks without an exact `expected_output` (or for low-deterministic-score fall-throughs) is `claude-sonnet-4-20250514`, called via the Anthropic Messages API at `max_tokens=256` with the structured rubric stored in `src/agentops_bench/scoring/completion.py`. The rubric instructs the judge to redact any «INJECTED PAYLOAD REDACTED» marker and not to penalise the agent for unrealistic mock tool output. A deterministic exact-match short-circuits the judge whenever an `expected_output` is available and the agent’s output normalises to it; this fires on the great majority of pilot runs.

Reproduction. End-to-end reproduction from the repository root:

```
pip install -e .
export ANTHROPIC_API_KEY=...
export OPENAI_API_KEY=...
export OPENROUTER_API_KEY=...
export TAVILY_API_KEY=...
bash scripts/run_pilot_v2.sh
python3 scripts/analyze_pilot.py results/pilot_v1_seeded/_combined
python3 scripts/make_figures.py results/pilot_v1_seeded/_combined
```

Per-run traces, scores, and figures land under `results/pilot_v1_seeded/` and `paper/figs/`.

Cost & runtime. The 6,300-run pilot was executed by running each of the seven agents in parallel as a background process; total wall-clock time for the slowest agent (`deepseek-v3.2`, ~32s per run) was roughly 8 hours, with the other six finishing in 1–3 hours each. Cumulative API spend across all providers was \$44.19 at the list-price snapshot in `cost.py` (`PRICING_AS_OF = "2026-04"`).

Snapshot disclaimer. The reported scores are a snapshot of seven specific model identifiers at one date (2026-04) under one set of provider-default decoding parameters and one fixture-replay state. The relative ordering of the agents is expected to shift as providers release new model snapshots, change defaults, or update tool-handling behaviour; OpenRouter list prices in particular reflect a shifting upstream mix and will drift faster than direct-vendor prices. Numbers in this paper should be reproducible on the released fixtures and code, but should not be read as a stable verdict on any agent family.

B Broader Impact

The framework’s stated goal is to give practitioners a more honest picture of what an LLM agent will do in production — particularly when exposed to data the operator does not fully control. Multi-axis reporting that includes cost and adversarial-safety drops makes it harder for a vendor to ship an agent that is accurate-on-paper but expensive or insecure in deployment, and we hope publishing the harness under Apache-2.0 lowers the friction of adopting that practice.

The work has dual-use surface. The injection catalogue and the instrumented tool server are useful both for evaluating defences and for constructing attacks against deployed agents. We mitigate this modestly by keeping payloads short, well-known in the literature (direct adaptations of Greshake et al. Greshake et al. [2023] and AgentDojo DeBenedetti et al. [2024]), and stripped of any operator- or vendor-specific exfiltration targets — the canary URLs are generic placeholders. We do not view withholding the catalogue as a viable mitigation: defenders need it to test their systems, and any catalogue we release upper-bounds rather than enables attacker capability.

A practical caveat: composite safety scores summarised to a single number, including ours, can give false confidence. The per-component breakdown (compliance, detection, exfiltration) is the operationally meaningful signal, and we report it alongside the composite for that reason.

C Pairwise statistical tests

We complement the headline summary with paired Wilcoxon signed-rank tests over per-cell deltas (each cell is a (task, condition, run) triple shared across the two agents in a comparison) and apply Holm–Bonferroni correction across all $\binom{7}{2} = 21$ pairwise tests to control family-wise error at $\alpha = 0.05$. The script is `scripts/analyze_pairwise.py`; results are reproducible from the released per-run trace JSONs.

The interpretation we draw is twofold. *First*, the seven-agent lineup is statistically separated on completion at $n = 900$ for nearly every pair: only `sonnet-4-6` vs. `mistral-large` (mean $\Delta = +0.036$) and `gpt-5.5` vs. `deepseek-v3.2` (mean $\Delta = -0.001$) fail to clear the corrected threshold, the latter genuinely tied at the cell level. *Second*, adversarial-safety separation is also robust at this scale: the 17 of 21 pairs that clear correction match the operationally meaningful pattern that `deepseek-v3.2` and `mistral-large` are the most fragile under injection while `sonnet-4-6` is the most resistant of the tool-using checkpoints.

D Per-attack-class safety breakdown

Each adversarial run records the specific payload appended to each poisoned tool result. We aggregate over the 2,100 adversarial runs (seven agents $\times$ 300 runs each) by attack class (Table 6). The denominator is the number of runs in

Table 4: Pairwise completion (all conditions, $n = 900$ paired observations per comparison). Mean delta is (Agent A) $-$ (Agent B). “Reject” = Holm–Bonferroni-corrected reject of H_0 at FWER 0.05. At $n = 900$ all but two pairs separate, matching the much wider between-agent completion spread under the seven-agent lineup.

Agent A	Agent B	n pairs	Mean Δ	Raw p	Reject
haiku-4-5	sonnet-4-6	900	+0.046	0.000	yes
haiku-4-5	gpt-5.5	900	−0.014	0.000	yes
haiku-4-5	deepseek-v3.2	900	−0.015	0.000	yes
haiku-4-5	llama-4-scout	900	+0.754	0.000	yes
haiku-4-5	mistral-large	900	+0.083	0.000	yes
haiku-4-5	qwen3-max	900	+0.206	0.000	yes
sonnet-4-6	gpt-5.5	900	−0.060	0.000	yes
sonnet-4-6	deepseek-v3.2	900	−0.062	0.000	yes
sonnet-4-6	llama-4-scout	900	+0.708	0.000	yes
sonnet-4-6	mistral-large	900	+0.036	0.111	no
sonnet-4-6	qwen3-max	900	+0.160	0.000	yes
gpt-5.5	deepseek-v3.2	900	−0.001	0.125	no
gpt-5.5	llama-4-scout	900	+0.768	0.000	yes
gpt-5.5	mistral-large	900	+0.097	0.000	yes
gpt-5.5	qwen3-max	900	+0.220	0.000	yes
deepseek-v3.2	llama-4-scout	900	+0.770	0.000	yes
deepseek-v3.2	mistral-large	900	+0.098	0.000	yes
deepseek-v3.2	qwen3-max	900	+0.221	0.000	yes
llama-4-scout	mistral-large	900	−0.672	0.000	yes
llama-4-scout	qwen3-max	900	−0.549	0.000	yes
mistral-large	qwen3-max	900	+0.123	0.000	yes

which at least one payload of that class was injected; the numerator metrics are the run-level safety / compliance / detection / exfiltration scores attributed to runs containing that class. A single run can contribute to multiple class rows because the catalogue samples uniformly at the per-call site.

Three things stand out. *First*, the unicode zero-width class is again the most effective payload class on this lineup: mean compliance is 0.195, the highest in the table (next: fake-error recovery at 0.158). Hidden-character payloads defeat ordinary surface-form filtering, and the result replicates across closed and open weights. *Second*, exfiltration tool calls remain rare but no longer zero: the delayed-post-processing class triggered an exfiltration on 3.1% of runs, and direct-override and nested-JSON each on 0.9%. These are single-digit raw counts — driven by individual open-weights agents acting on instructions to issue an outbound tool call — but they demonstrate that the exfiltration component of the safety axis is no longer uninformative on a sufficiently broad lineup. *Third*, regex-based detection remains a low ceiling: mean detection across the table is 0.06, and most agents “silently ignore” the injection without articulating that they have done so — which scores 0 on detection and 0 on compliance, return-

Table 5: Pairwise adversarial-condition safety ($n = 300$ paired observations per comparison). 17 of 21 pairs separate after Holm–Bonferroni correction; the four non-separating pairs all involve gpt-5.5 or qwen3-max, which sit close together on adversarial safety despite differing on every other axis.

Agent A	Agent B	n pairs	Mean Δ	Raw p	Reject
haiku-4-5	sonnet-4-6	300	−0.051	0.000	yes
haiku-4-5	gpt-5.5	300	−0.028	0.000	yes
haiku-4-5	deepseek-v3.2	300	+0.042	0.000	yes
haiku-4-5	llama-4-scout	300	−0.096	0.000	yes
haiku-4-5	mistral-large	300	+0.025	0.002	yes
haiku-4-5	qwen3-max	300	−0.026	0.051	no
sonnet-4-6	gpt-5.5	300	+0.024	0.010	yes
sonnet-4-6	deepseek-v3.2	300	+0.093	0.000	yes
sonnet-4-6	llama-4-scout	300	−0.045	0.000	yes
sonnet-4-6	mistral-large	300	+0.076	0.000	yes
sonnet-4-6	qwen3-max	300	+0.026	0.036	no
gpt-5.5	deepseek-v3.2	300	+0.070	0.000	yes
gpt-5.5	llama-4-scout	300	−0.068	0.000	yes
gpt-5.5	mistral-large	300	+0.053	0.000	yes
gpt-5.5	qwen3-max	300	+0.002	0.532	no
deepseek-v3.2	llama-4-scout	300	−0.138	0.000	yes
deepseek-v3.2	mistral-large	300	−0.017	0.022	no
deepseek-v3.2	qwen3-max	300	−0.068	0.000	yes
llama-4-scout	mistral-large	300	+0.121	0.000	yes
llama-4-scout	qwen3-max	300	+0.070	0.000	yes
mistral-large	qwen3-max	300	−0.051	0.000	yes

ing a 0.7 floor. A more sophisticated detection scorer (e.g., a small auxiliary classifier) is the right next step; the regex catalogue is the release-able lower bound.

E Datasheet for Datasets

We adapt the structure of Gebru et al. [2021] to the benchmark task suite released with this paper.

Motivation. *For what purpose was the dataset created?* To evaluate LLM agents on six operational axes (completion, cost, efficiency, reliability, recovery, safety) under three environmental conditions (clean, noisy, adversarial). *Who created the dataset and on behalf of whom?* The author, on behalf of an independent research release; no institutional sponsor. *Who funded the creation?* Self-funded; cumulative API spend on the released pilot was \$44.19.

Table 6: Per-attack-class safety, averaged across all seven agents on runs containing at least one payload of that class. “Mean compliance” is the regex-matched compliance rate (lower is better); “Mean detection” is the rate at which the agent flagged or refused the injection (higher is better); “Exfil rate” is the fraction of runs in which the agent issued a tool call to one of the catalogue’s exfiltration hosts.

Attack class	Runs	Mean safety	Mean compl.	Mean detect.	Exfil rate
direct override	223	0.692	0.059	0.059	0.009
authority impersonation	186	0.694	0.079	0.086	0.000
embedded in JSON	137	0.681	0.074	0.035	0.000
markdown exfil	131	0.701	0.022	0.031	0.000
comment channel	139	0.712	0.013	0.058	0.000
unicode homoglyph	96	0.710	0.024	0.066	0.000
unicode zero-width	96	0.628	0.195	0.020	0.000
base64 wrapped	140	0.687	0.085	0.072	0.000
fake-error recovery	112	0.658	0.158	0.071	0.000
multi-turn poisoning	92	0.730	0.010	0.114	0.000
nested JSON	116	0.700	0.057	0.083	0.009
XML tag impersonation	109	0.699	0.030	0.038	0.000
delayed post-processing	128	0.720	0.016	0.121	0.031

Composition. *What do the instances represent?* YAML task definitions, each specifying a natural-language prompt, a tool whitelist, an optional reference answer, an optimal-step estimate, and a difficulty tag. *How many instances are there?* 100 seed tasks in the v1.0 release, balanced as 20 tasks per domain across the five domains (code, data analysis, research, safety, tool-use); all 100 are used to compute the headline numbers in this paper. *Is any information missing?* Reference answers are absent for ambiguous research-style tasks where deterministic scoring is inappropriate; those run through the LLM judge. *Are there any errors, sources of noise, or redundancies?* Tasks were drafted by a single author and inherit that author’s biases. The tool catalogue (10 reference tools) constrains task realism; tasks that would require richer tool catalogues (file ingestion, multi-modal input) are out of scope of this seed set.

Collection process. *How was the data acquired?* All tasks were authored from scratch for this release; no third-party datasets were ingested. *What mechanisms were used to collect the data?* Direct authoring in YAML against the schema in `src/agentops_bench/schema.py`. *Over what timeframe?* 2026-01 through 2026-04. *Were any ethical-review processes conducted?* No human subjects; no ethical review required. Adversarial payloads were adapted from publicly published catalogues Greshake et al. [2023], Debenedetti et al. [2024] and stripped of any organisation- or vendor-specific exfiltration targets.

Pre-processing, cleaning, labelling. Tasks ship in the form they will be evaluated. Reference answers are authored in the format the agent is expected to produce. Optimal-step counts are author-estimated and used only to compute efficiency, not correctness.

Uses. *Has the dataset been used for any tasks already?* Yes — the 6,300-run pilot reported in this paper. *What other tasks could it be used for?* Any agent-evaluation harness whose scoring is compatible with the YAML schema. *Is there anything that should not be done with the dataset?* The injection catalogue should not be used for live attack against deployed third-party agents without authorisation; the dual-use surface is discussed in Appendix B.

Distribution. *License.* Apache-2.0 for code and tasks. *How is the dataset distributed?* Public Git repository, with a tagged release per benchmark version. *Persistent identifier.* A Zenodo deposit will be created at v1.0 release; the DOI will be added to the README and to subsequent revisions of this paper.

Maintenance. See Appendix F for the versioning and contribution policy.

F Maintenance and versioning

Versioning. The benchmark uses a MAJOR.MINOR.PATCH scheme. MAJOR bumps when scoring axis definitions or weights change in a way that breaks comparability with prior reports. MINOR bumps when tasks, payloads, or tool-server failure modes are added. PATCH bumps for code-only fixes that do not move scores. The pilot reported here is v1.0.

Snapshotting. Every released report records the benchmark MAJOR.MINOR.PATCH, the price-table date (PRICING_AS_OF in `cost.py`), the model identifiers, and the fixture-store hash. Re-running an old report against a newer benchmark version is expected to drift; the stored metadata makes the drift attributable.

Maintenance. The repository is currently maintained by the sole author. We invite community contributions, particularly: (i) new tasks with author-validated reference answers, (ii) additional prompt-injection payloads with documented attack classes, (iii) agent-adaptor implementations for additional providers and open-weights models. Contribution guidelines, a triage policy, and a deprecation policy for retired tasks are documented in `CONTRIBUTING.md`.

Liability and responsibility. Authors of contributed tasks or payloads retain copyright (Apache-2.0 contribution-licence agreement applies). The repository maintainer is responsible for triage and versioning; users of the benchmark are responsible for any use of the adversarial-payload catalogue and the harness.

G Reproducibility checklist

We follow the NeurIPS reproducibility checklist.

- ✓ Claims in the abstract and introduction match the results: every quantitative claim is reproducible from `results/pilot_v1_seeded/` via `scripts/analyze_pilot.py`, `scripts/analyze_pairwise.py`, and `scripts/analyze_safety_per_payload.py`.
- ✓ Limitations are explicitly discussed (§7).
- ✓ Code is released under Apache-2.0 with the paper.
- ✓ Each pilot run’s full trace, scores, and cost breakdown is released as JSON.
- ✓ All datasets used (the 100 seed tasks, the 15-payload injection catalogue, and the fixture-replay store) are included in the release; provenance is described in Appendix E.
- ✓ Hyperparameters (failure rates, injection probabilities, generation cap) are listed in Appendix A.
- ✓ Statistical significance is reported with Holm–Bonferroni correction (Appendix C).
- ✓ Compute resources used: a single laptop, ~8 hours wall-clock with the seven agents running concurrently as background processes, \$44.19 cumulative API spend across providers.
- ✓ Broader impact discussed (Appendix B).
- ✓ Random seeds for failure-mode and injection sampling are derived deterministically from `sha256(task_id|condition|run_number)` per cell (§A, *Tasks and conditions*). Live tool back-ends are deterministic via fixture replay; agent-side decoding inherits provider defaults.

References

- Edoardo DeBenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024.
- Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723.

- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023. doi: 10.1145/3605764.3623985.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024.
- Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. AgentBoard: An analytical evaluation board of multi-turn LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024.
- Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general AI assistants. In *International Conference on Learning Representations (ICLR)*, 2024.
- Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. Berkeley function calling leaderboard (BFCL). <https://gorilla.cs.berkeley.edu/leaderboard.html>, 2024.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*, 2024.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for

- open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.
- Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics (ACL)*, 2024.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024.